Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторна робота № 5

« Нормалізація бази даних »

Роботу виконали

Студент групи ІО-45

Соловійщук

Назар

Роботу перевірив:

Русінов В.В.

Київ 2026

ЛАБОРАТОРНА РОБОТА № 5

Тема: Нормалізація бази даних

Мета:

Проаналізувати поточну схему бази даних на наявність надлишковості та аномалій.
Визначити функціональні залежності для основних таблиць.
Провести нормалізацію схеми до третьої нормальної форми (3NF) для покращення цілісності даних та усунення транзитивних залежностей.

SQL-скрипт

```
-- 1. Створюємо таблицю-довідник для жанрів
CREATE TABLE genres (
    genre_id SERIAL PRIMARY KEY,
    genre_name VARCHAR(100) NOT NULL UNIQUE
);

-- 2. Наповнюємо її унікальними жанрами з твоєї таблиці movies
INSERT INTO genres (genre_name)
SELECT DISTINCT genre FROM movies;

-- 3. Додаємо нове поле для зв'язку в таблицю movies
ALTER TABLE movies ADD COLUMN genre_id INTEGER;

-- 4. Оновлюємо id жанрів у таблиці movies на основі назв
UPDATE movies m
SET genre_id = g.genre_id
FROM genres g
WHERE m.genre = g.genre_name;

-- 5. Видаляємо старе текстове поле і ставимо обмеження
ALTER TABLE movies DROP COLUMN genre;
ALTER TABLE movies ALTER COLUMN genre_id SET NOT NULL;
ALTER TABLE movies ADD CONSTRAINT fk_movie_genre
FOREIGN KEY (genre_id) REFERENCES genres(genre_id);
```

Аналіз функціональних залежностей

Таблиця

Первинний ключ: customer_id
ФЗ: customer_id → first_name, last_name, phone, email, address, registration_date.
Висновок: Усі атрибути залежать лише від ключа, аномалій немає.

Таблиця movies (Кандидат на нормалізацію)

Первинний ключ: movie_id
Поточні ФЗ: movie_id → title, release_year, director, duration, genre.

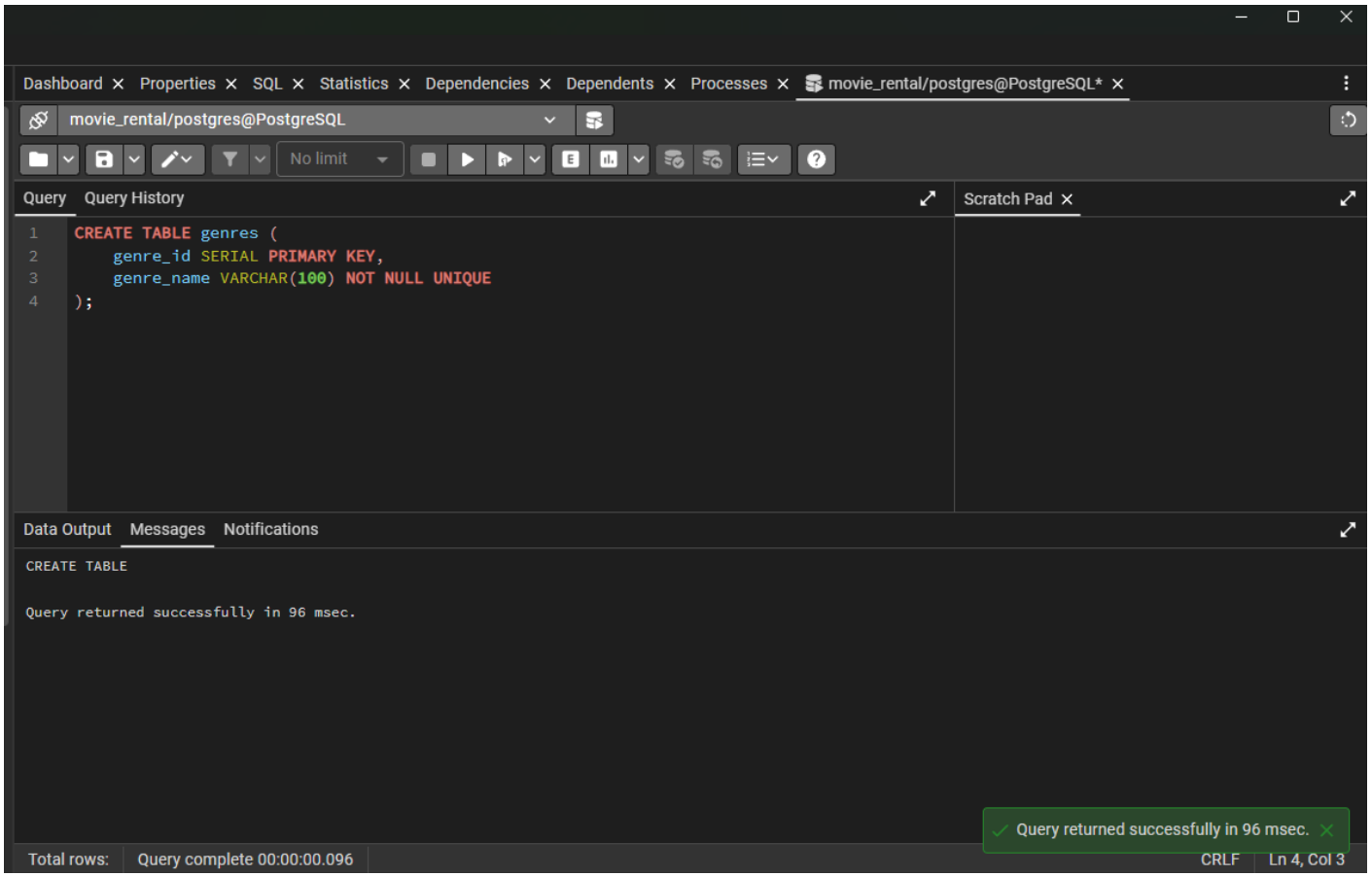
Проблема: Поле genre містило текстові назви. Оскільки один і той самий жанр повторюється для багатьох фільмів, виникає надлишковість та аномалія оновлення (при зміні назви жанру потрібно було б редагувати багато рядків).

3. Перевірка нормальних форм

1NF (Перша нормальна форма): Усі значення в таблицях атомарні. Виконується.
2NF (Друга нормальна форма): Усі неключові атрибути повністю залежать від первинного ключа (немає часткових залежностей від частин складеного ключа). Виконується.
3NF (Третя нормальна форма): Було виявлено транзитивну залежність: movie_id → genre_name. Насправді жанр — це окрема сутність. Для досягнення 3NF необхідно винести жанри в окрему таблицю-довідник.

4. Процес нормалізації

Для усунення надлишковості було проведено декомпозицію таблиці movies:
Створено нову таблицю genres (genre_id, genre_name).
Текстове поле genre у таблиці movies замінено на зовнішній ключ genre_id.
Встановлено зв'язок: movies.genre_id → genres.genre_id.
Нові функціональні залежності:
movie_id → title, release_year, director, duration, genre_id
genre_id → genre_name



Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
INSERT INTO genres (genre_name)
SELECT DISTINCT genre FROM movies;
```

Data Output Messages Notifications

```
INSERT 0 2

Query returned successfully in 42 msec.
```

Query returned successfully in 42 msec.

Total rows: Query complete 00:00:00.042 CRLF Ln 2, Col 35

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
ALTER TABLE movies ADD COLUMN genre_id INTEGER;
```

Data Output Messages Notifications

```
ALTER TABLE

Query returned successfully in 32 msec.
```

Query returned successfully in 32 msec.

Total rows: Query complete 00:00:00.032 CRLF Ln 1, Col 48

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
UPDATE movies m
SET genre_id = g.genre_id
FROM genres g
WHERE m.genre = g.genre_name;
```

Data Output Messages Notifications

```
UPDATE 5

Query returned successfully in 34 msec.
```

Query returned successfully in 34 msec.

Total rows: Query complete 00:00:00.034 CRLF Ln 4, Col 30

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

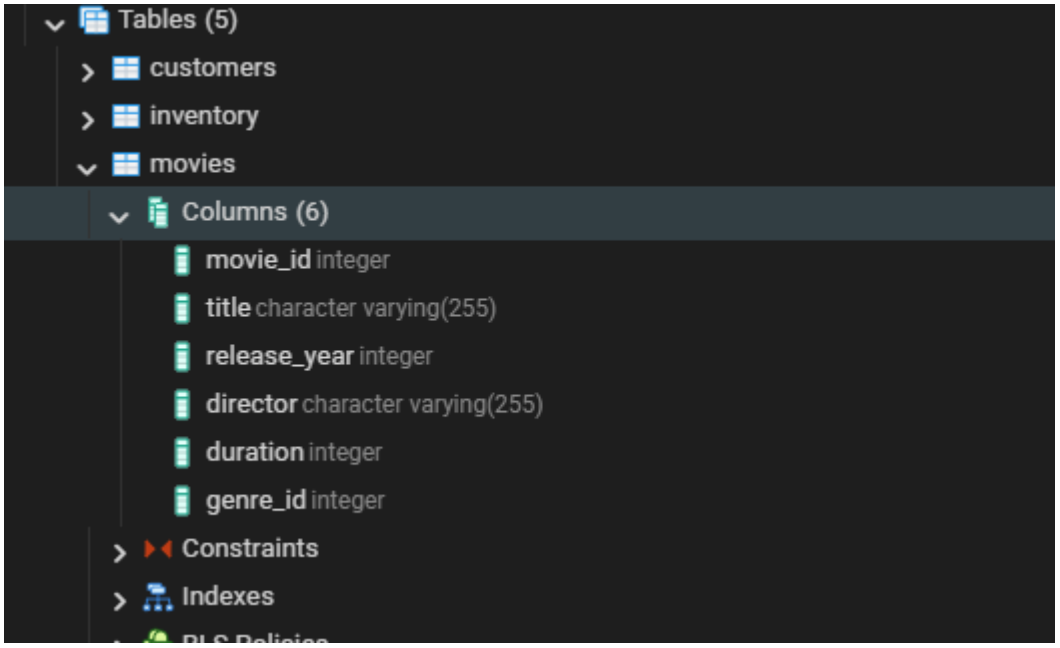
```sql
ALTER TABLE movies DROP COLUMN genre;
ALTER TABLE movies ALTER COLUMN genre_id SET NOT NULL;
ALTER TABLE movies ADD CONSTRAINT fk_movie_genre
FOREIGN KEY (genre_id) REFERENCES genres(genre_id);
```

Data Output Messages Notifications

```
ALTER TABLE

Query returned successfully in 35 msec.
```

Total rows: Query complete 00:00:00.035 CRLF Ln 4, Col 52


Tables (5)
customers
inventory
movies
Columns (6)
movie_id integer
title character varying(255)
release_year integer
director character varying(255)
duration integer
genre_id integer
Constraints
Indexes

Висновок:
У ході роботи було проведено реорганізацію схеми бази даних системи прокату фільмів. Завдяки винесенню жанрів в окрему таблицю-довідник було усунуто дублювання текстових даних. Це дозволило привести базу даних до третьої нормальної форми (3NF), забезпечивши вищу цілісність даних та спростивши подальше обслуговування бази.